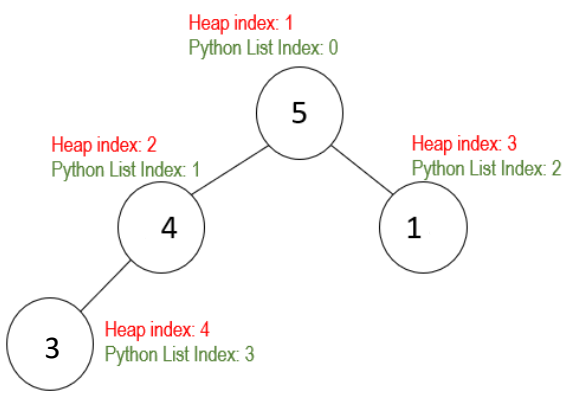
Lab 7 (10 points) **Due date:** December 1st, 2023, 11:59 PM

Before you start…

Keep in mind these tips from your instructor:

- Watch the lectures in Module 6.1 before you start this assignment
- Do not compare yourself to others: An assignment might take you hours, compared to other fellow students, but every student learns in a different way. Please reach out to the course staff id you are stuck and need help
- Outline a strategy before you start typing code and share it with the course staff for additional support
- Note that the index passed into the `_parent`, `_leftChild` and `_rightChild` methods are 1-indexed (as heap indices usually are), but a Python list is 0-indexed. Make sure to subtract 1 in your final index to make up for that difference when accessing elements in self._heap. Do not leave an empty space in the 0th index of `self._heap`.




```
>>> x = MaxBinaryHeap()
>>> x._heap = [5, 4, 1, 3]  # force load a maximum heap
>>> x._parent(4)  # Heap index 4 (value: 3) has a parent at index
4                 # 4//2 =2, which is list index 1 (x._heap[1])
>>> x._leftChild(2)  # Heap index 2 (value: 4) has a left child at index
3                  # 2 * 2 = 4, which is list index 3 (x._heap[3])
>>> x._rightChild(1) # Heap index 1 (value: 5) has a right child at index
1                  # 2 * 1 + 1 = 3, which is list index 2 (x._heap[2])
```

- Ask questions using our Lab 7 channel in Microsoft Teams

REMINDER: As you work on your coding assignments, it is important to remember that passing the examples provided does not guarantee full credit. While these examples can serve as a helpful starting point, it is ultimately your responsibility to thoroughly test your code and ensure that it is functioning correctly and meets all the requirements and specifications outlined in the assignment instructions. Failure to thoroughly test your code can result in incomplete or incorrect outputs, which will lead to deduction in points for each failed case.

Create additional test cases and share them with you classmates on Teams, we are in this together!

90% > Passing all test cases 10% > Clarity and design of your code

The MaxBinaryHeap class **(8 pts)**

As discussed in the first part of module 6, a binary heap is a complete binary tree that satisfies the heap ordering property and can be implemented with an array. In this assignment, you will be implementing a maximum binary heap using a Python list

Attributes

Type	Name	Description
list	_heap	A list containing the elements of the binary heap

Methods

Type	Name	Description
int or float	_parent(self, index)	Gets the value of the parent of the node at an index
int or float	_leftChild(self, index)	Gets the value of the left child of the node at an index
int or float	_rightChild(self, index)	Gets the value of the right child of the node at an index
None	insert(self, item)	Inserts an item to the maximum heap
int,float	getMax(self)	Gets the maximum value in the heap
int or float	deleteMax(self)	Removes the maximum element of the heap

Special methods

Type	Name	Description
str	__repr__(self), __str__(self)	String representation of this object
int	__len__(self)	The number of elements in the heap

NOTE: Your code must use the formulas given in the lectures (k//2, 2*k, 2*k+1). No credit will be given if you use other formulas!

_parent(self, index) **(0.25 pts)**

Returns the value of the parent of the element at the specified index. Please see the notes on page 1 about heap and list indices. c

Input		
int	index	The heap index to find a child of

Output	
int or float	Value of that element's parent
None	None is returned if that element does not have a parent

_leftChild(self, index) and _rightChild(self, index) **(0.25 pts each)**

Returns the value of the left/right child of the element at the specified index. Please see the above note about heap and list indices. You cannot use try/except blocks.

Input		
int	index	The heap index to find a child of

Output	
int or float	Value of that element's left/right child
None	None is returned if that element does not have the specified child

getMax(self) **(0.25 pts)**

A property method that returns the maximum value in the heap. You are not allowed to use the max method or any sorting methods.

Output	
int or float	The maximum value in the heap
None	None is returned if the heap is empty

insert(self, item) **(2 pts)**

Inserts an item into the heap while maintaining the maximum heap property. It must use the _parent(self, index) method, so make sure to test it before using it in this method.

Input		
int or float	item	The value to add to the heap

deleteMax(self) **(3 pts)**

Removes the maximum element of the heap and returns the value of the element that was just removed. The special cases where the heap is empty or contains only one element have been implemented for you already, your task is to implement the general case. As a reminder, according to the maximum heap property, the maximum element is the root node. When percolating down, if both children have the same value, swap (if needed) with the **left child**. It must use the _leftChild and _rightChild methods, so make sure to test them before using them in this method

Output	
int or float	The maximum value of the heap before deletion

__len__(self)

Gets the number of elements in this heap by overloading the len function. This method has already been implemented for you.

Output	
int	Number of elements in the heap

__repr__(self), __str__(self)

Gets the string representation of this object. These methods have already been implemented for you.

Output	
str	Representation of this object as a string.

Additional Grading Functionality **(2pts)**

The grading script will perform a series of mixed heap operations and compare the final status of your heap. The last 2 points are based on the correctness of the heap after the mixed operations are completed. Verify that all methods work correctly when method calls are performed in a mixed and random manner. **There is no partial credit if the final heap is incorrect.**

Section 2: The heapSort function

As discussed in our video lectures in module 7, heap sort is a comparison-based sorting technique based on the Binary Heap data structure. It builds a heap with the given list and removes the root node until the heap is empty, producing a sorted list. Using your code from Section 1, implement the function heapSort that takes a list of numbers and returns a new list that is sorted.

As a reminder, you are not allowed to use the sorted() method or the sort operator. Your code will not get credit if you use them. Your function must use an instance of the MaxBinaryHeap class to complete the sorting process.

heapSort(numList) **(2 pts)**

Takes a list of numbers and uses the heap sort algorithm to return a new list with the elements of numList sorted in descending order. It does NOT perform an in-place sorting, this means, your code must return a new sorted list without modifying the original one. You are only allowed to use the MaxBinaryHeap operations described in this assignment (len, insert and deleteMax).

Input		
list	numList	A sequence of numerical values. You cannot make assumptions about its size

Output	
list	Values of numList sorted in descending order using the heap sort algorithm